

**OptiCode : AI-Driven adaptive Coding Assistance and
Interview Preparation Platform**

25-26J-364

Project Proposal Report

G.V.G.A.S.Hettiarachchi– IT22604194

B.Sc. (Hons) in Information Technology Specializing in
Information Technology

Department of Computer Science & Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

August 2025

**OptiCode : AI-Driven adaptive Coding Assistance and
Interview Preparation Platform**

25-26J-364

Project Proposal Report

B.Sc. (Hons) in Information Technology Specializing in
Information Technology

Department of Computer Science & Information Technology

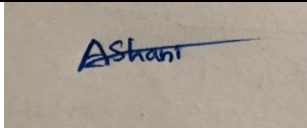
Sri Lanka Institute of Information Technology

Sri Lanka

August 2025

DECLARATION

We declare that this is our own work, and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Name	Student ID	Signature
G.V.G.A.S.Hettiarachchi	IT22604194	

The supervisor/s should certify the proposal report with the following declaration.

The above candidates are carrying out research for the undergraduate Dissertation under my supervision.



31/08/2025

Signature of the supervisor:

Date

ABSTRACT

The problem that is usually faced by university students is how to bridge the gap between what they learn in theory and what they need to learn practically in the form of the software industry. Although current Intelligent Tutoring Systems (ITS), AI-based code generation tools and current IDEs will offer syntax correction or other investigative features, they do not offer the provision of integrated, adaptive and long-term assistance that identify the weaknesses that occur and equip students to navigate the reality of an AI-driven job search. The main research question is how to create a single, smart learning platform that does not only identify mistakes in coding in real time but also offers conceptual clarifications, adaptability in learning reinforcement, and career-related interview training.

To solve this, the research suggests an AI-Powered Web Platform to Code Concept Extraction, Code Refactoring, Adaptive Real-Time Weakness Detection with Guided Learning and Question Generation, and AI-Powered Interview Simulation. The system combines four modules: a Code Concept Extractor mapping written code to theoretical concepts with NLP and static analysis; a Code Refactoring Engine to provide intelligent and explainable readability and performance improvements; an Adaptive Weakness Detection module to use Abstract Syntax Tree (AST) monitoring, customized Weakness Profiles and even gamified micro-challenges; and an Interview Simulator to generate personalized coding and theory questions with detailed feedback. The methodology will be iterative in design and evaluation that will entail the pre/post assessment, learning analytics, and controlled user evaluation to confirm the effectiveness of the system.

The expected results are enhanced coding productivity, enhanced conceptual knowledge, long-term motivation with gamification and quantifiable technical interview and industry-level preparedness. The study brings innovative and integrated solution that serves as a continuous, AI-driven coding tutor that grows alongside the student development and helps them to move out of academia and into industry in an increasingly AI-focused world.

KEYWORDS

Programming education gap, AI-Powered Web Platform, Code Concept Extraction, Code Refactoring, Adaptive Real-Time Weakness Detection & Guided Learning, AI-Powered Interview Simulation, AI-driven job preparedness, Adaptive Learning

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
ITS	Intelligent Tutoring Systems
IDE	Integrated Development Environment
AST	Abstract Syntax Tree
NLP	Natural Language Processing

Table 1: List of Abbreviations

Table of Contents

DECLARATION	i
ABSTRACT	ii
LIST OF ABBREVIATIONS	iii
1. INTRODUCTION.....	1
1.1 Background and Literature	1
1.2 Research Gap.....	3
1.3 Research Problem	4
2. OBJECTIVES	4
2.1 Main Objective	5
2.2 Specific Objectives	5
3. METHODOLOGY	6
3.1 Requirement Gathering.....	6
3.1.1 Past Research Analysis.....	7
3.2 Development Methodology.....	7
3.3 Project Management.....	8
3.4 Feasibility Study	10
3.4.1 Technical Feasibility.....	10
3.4.2 Schedule Feasibility	10
3.4.3 Economic Feasibility	11
3.5 System Analysis.....	12
3.5.1 Software solution approach	12
3.5.3 Tools and technologies	14
3.6 Project requirements	15

3.6.1	Functional requirements	15
3.6.2	Non- Functional requirements	16
3.6.3	User requirements.....	
3.7	Project scope.....	20
3.7.1	In-scope.....	20
3.7.2	Out-scope	21
3.8	Testing.....	22
3.8.1	Unit Testing.....	22
3.8.2	Integration Testing.....	22
3.8.3	Usability Testing	22
3.8.4	Functional Testing	23
3.8.5	Accessibility Testing	23
3.8.6	Performance Testing.....	23
4.	WORK BREAKDOWN.....	24
4.1	Time Line	25
5.	COMMERCIALIZATION.....	26
6.	BUDGET.....	27
7.	REFERENCE	28
8.	APPENDICES.....	29
8.1	Plagiarism Report... ..	29

List of Tables

Table 1:	List of Abbreviations	iii
Table 2:	Competitive Analysis	3
Table 3:	Use case scenario1	19
Table 4:	Use case scenario2	20
Table 5:	Use case scenario3	20

Table 6: Cost management plan.....	27
------------------------------------	----

List Of Figures

Figure 1: Agile Methodology	9
Figure 2: System Diagram	13
Figure 3: Overall System Diagram	14
Figure 4: Work breakdown structure	24
Figure 5: GanttChart.....	25

1. INTRODUCTION

1.1 Background and Literature

An old problem of programming education has been the trade-off between teaching theories and learning practical skills. University programs usually present fundamental concepts of recursion, data structure and object-oriented programming (OOP). Nevertheless, not all students are able to translate this theoretical knowledge into maintainable, efficient, and industry ready code [1],[2]. Recursion is difficult to learn as one example: to do recursion, a student must think abstractly, and base cases must be handled carefully.

Modern Integrated Development Environments (IDEs), such as Visual Studio Code and IntelliJ IDEA, provide students with helpful features like syntax highlighting, auto-completion, and error detection. Although these features increase efficiency in code, they emphasize syntax-level problems and fail to give feedback based on the conceptual limitations or recurrent limitations of a student [3]. Students can therefore produce code that is correct yet inefficient, bad practices or a misconception of the underlying forms.

At the same time, the software engineering field is experiencing a dramatic change as developer tools that are powered by AI, such as GitHub Copilot or ChatGPT, are currently in use. Such tools may produce high volumes of code, which may drive down the entry-level developer workforce that handles more mundane code-writing [4], [5]. There is a resulting movement in industry toward higher-order skills, including code analysis, optimization, and problem-solving, where engineers test, refine and improve their own and AI-generated code. This change reveals the significance of instruction in conceptual, and not only functional, mastery of students.

It has suggested the use of Intelligent Tutoring Systems (ITS) that can resolve these educational problems. It has been demonstrated that individualized feedback can make a massive difference in student learning [6]. Most ITS solutions, however, concentrate on

supplying hints as individual tasks and do not track long-term learner profiles. Likewise, AI-based coders like GitHub Copilot or SonarQube provide smart suggestions and code quality checks, and are not teaching tools but not adapted pedagogically to the level of student knowledge [4], [5].

In order to create a proper solution, a number of technical and educational plans have to be learned: Program analysis in Abstract Syntax Tree (AST): to keep track of patterns of code and identify repeating errors and to measure student coding behavior. Adaptive methods of learning: To dynamically adapt difficulty of tasks and the feedback based on the skill level of a student. Gamification plans: to keep motivated and interested with the help of the elements of the game.

1.2 Research Gap

Though IDEs and AI assisted coding tools give students immediate feedback, they do not encourage conceptual knowledge or monitor recurring weakness. The current ITS systems are configured to represent certain activities and can seldom retain a long-term record of student mistakes that can be used to shape future learning. Moreover, the existing methods fail to equip students with the AI-based transformation of the employment landscape, where the process of coding might be automated, however, more advanced thinking and optimization capabilities cannot be neglected. Thus, the loophole is in the fact that there is no education platform capable of:

1. Track student coding activity,
2. Adapt feedback to individual skill levels,
3. Keep long-term records of chronic weaknesses,
4. Learn to reinforce gamified.

This is the gap of not only technical (AST-based monitoring, adaptive systems) but also pedagogical (personalized learning, skill reinforcement) issues that does not have a full solution provided by current tools.

Tool/Platform	Syntax Correction	Conceptual Feedback	Adaptive Learning	Long-term Tracking	Gamification	Notes
Visual Studio Code	YES	NO	NO	NO	NO	Good for syntax & debugging
IntelliJ IDEA	YES	NO	NO	NO	NO	Like VS Code, strong refactoring support
GitHub Copilot	YES	NO	NO	NO	NO	AI code suggestions, not pedagogically adaptive
SonarQube	YES	NO	NO	NO	NO	Code quality feedback only
Existing ITS (Code Tutor)	YES	YES	Partial	NO	NO	Provides hints, but no long-term learner profile

Table 2: Competitive Analysis

1.3 Research Problem

AI has changed the software engineering industry by its capability to create and optimize code. It is an indication that as routine coding functions, possibly, become automated, problem-solving and conceptual skills are in higher demand than ever. In the absence of these higher order skills, students stand chances of being unprepared to this new reality. There is a big disparity in what the current tools provide and what students require to achieve success in this dynamic environment, as indicated in Table 2.

In particular, it is the following limitations of the existing solutions that define the research problem:

Absence of Conceptual Feedback: Software such as Visual Studio Code, IntelliJ IDEA and GitHub Copilot are good at correcting syntax and debugging code, but do not provide conceptual feedback, as Table 2 demonstrates. This implies that a student is able to correct a syntax error without the knowledge of the theoretical concept behind it, which results into repetition of conceptual errors.

None Adaptive Learning or Long term Tracking: The current tools, such as the existing Intelligent Tutoring Systems (ITS) do not have adaptive learning and the long term tracking process. This limits the detection of the repeated weaknesses of the student as well as the individualization of the learning process. In the absence of this functionality, students would not be directed toward mastery and would make the same mistakes repeatedly.

Lack of Gamification: As is observed in Table 2, all the analyzed tools do not involve any gamification to keep their students motivated. This puts students unprepared to the move to the industry where problem solving is as important as the ability to code. Here, the key research question is as follows: how can one single, integrated platform be created to fix not only the code, but also diagnose and fix the systemic conceptual weaknesses of a student with a personalized, adaptive, and gamified learning experience, and eventually make them marketable in an AI-enhanced world?

2. OBJECTIVES

2.1 Main Objective

To create and deploy an AI Teacher Module capable of offering adaptive real-time weakness identification with guided learning to allow university students to detect misconceptions in regularizing the use of Codes and to obtain personalized feedback as well as progressively ability their programming skills through interactive training.

2.2 Specific Objectives

1. **Weakness Detection**– Build an AST based monitor that identifies weak elements of code (e.g. recursion being used improperly, inefficient loops, wrong base cases) with at least 80% accuracy during live coding sessions.
2. **Adaptive Feedback**– Use a real-time guided feedback engine, which modulates the hints and explanations depending on the performance level of the student, which promotes progressive scaffolding but not generic solutions.
3. **Guided Learning**– Create an interface that permits the student to write their own coding questions, answer them, and be guided step-by-step to reproduce instead of being given direct answers, simulating the tutoring of a human teacher.
4. **Long-Term Profiling**– Develop a weakness tracking profile where the student improvement can be documented and where reinforcement exercises are added so that in 4 weeks of consistent use the student will have improved 30 percent in the areas of weakness identified.
5. **Gamified Engagement**– Consider the gamification features (levels, badges, progress tracking) to maintain a certain level of motivation, with the goal of achieving at least 70 percent engagement among the students throughout the practice sessions within a semester.

3. METHODOLOGY

3.1 Requirement Gathering

To make sure that the proposed platform addresses the needs of students, educators, and industry stakeholders, it will be necessary to collect the requirements. It will be done through the following approaches: Primary Requirements Interview a set of university students, and determine typical programming errors (e.g., recursion errors, inefficient code, false concepts). Collect feedback on the gaps in learning experienced in programming courses as perceived by lecturers. Gather expectations of new graduates and industry experts regarding interview expectations and real-world coding.

Additionally, Look at the documentation of Intelligent Tutoring Systems (ITS), AI-assisted code generation tools (e.g., GitHub Copilot, SonarQube) and e-learning. Compare course content of principal programming courses to determine areas of conceptual overlap that should be enhanced by the system.

The result of this phase is a requirements specification (containing functional requirements e.g. AST-based weakness detection, guided feedback, gamification, interview simulation) and non-functional requirements e.g. usability, reliability, scalability).

The results of this requirement gathering activity will directly guide the design of the AI Teacher Module, such as the creation of the AST-based weakness detection system, the design of the adaptive feedback engine, creation of tailored guided learning paths, and elements of gamification that ensures the platform is effective in helping to address student learning gaps and increase the acquisition of programming skills.

3.1.1 Past Research Analysis

The literature review will involve a profound examination of the previous studies to find out the existing gaps and best practices:

Intelligent Tutoring Systems (ITS): The effectiveness of the individual feedback has been shown to be effective but the absence of long term learner profiling is pointed out as a weakness.

Artificial Intelligence Aided Coding: research on GitHub Copilot and SonarQube demonstrates a positive effect on productivity but lacks pedagogical support of students.

Gamification in Education: The studies have found that gamification heightens student engagement and motivation, and most of them implement it at a surface level without changing to match the knowledge levels.

Adaptive Learning Models: Adaptive learning strategies have been shown to enhance conceptual learning; however they are not technically coupled with real-time code platforms. This discussion explains why AST-based code monitoring, adaptive feedback, gamification and AI-assisted interview training can be integrated into one platform.

3.2 Development Methodology

The AI-Powered Web Platform on Code Concept Extraction and Interview Preparation will be created in an iterative, modular way, a combination of agile and best practices of software engineering. The methodology will see to it that every component is created, tested and incorporated effectively and that it meets functional and non-functional requirement that were discovered in the process of requirement gathering.

Agile enables feedback, quick prototyping, incremental delivery, which best suits creating an AI-powered learning platform where usability and adaptability are critical.

The development will be arranged in time-boxed sprint (2- 3 weeks) and each sprint will cover one or more central modules of the platform. The first sprints will focus on the development of the Code Input Environment and the AST Analyzer that will become the basis of real-time weaknesses detection. Later sprints will concentrate on the Weakness Detection Engine, which will mean proper detection of coding errors, inefficiencies, and conceptual gaps.

The next phase of the Adaptive Feedback Module and Guided Learning Interface will provide students with personalized hints and step-by-step guidance based on their coding behavior and previous performance. Another facet being developed in parallel will involve the Progress Profiling and Tracking System to create long-term weakness profiles defining the way forward for adaptive learning.

As the final step, the Gamification Layer will be added to further engage students, motivating them with badges, points, and progress tracking.

Iterative improvement sits at the heart of the approach. At the end of each sprint, we will test and review prototypes and module outputs with students, supervisors, and other stakeholders. They will provide feedback on the design and usability that will allow us to redesign algorithms, amend adaptive processes, clarify the relevance of feedback, and improve user experience. This will make sure the system is designed to change with educational goals and needs of real-world usability.

3.3 Project Management

This project will be managed by standard project management principles adapted to the Agile framework to deliver on time, deliver quality outcomes, and inform team work which may involve collaboration across groups. The process of prioritizing tasks, tracking progress and managing risks will be the primary focus throughout the software development life cycle.

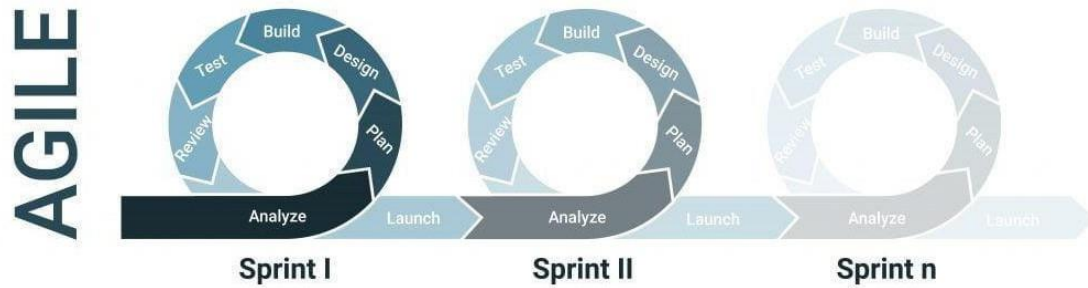


Figure 1: Agile Methodology

Take a development approach allowing tasks to be broken apart into sprint durations of 2–3 weeks. Each sprint will focus on a specific module, such as the AST Analyzer, Adaptive Feedback Engine, Guided Learning Interface, Progress Profiling, and Gamification Layer.

Each team member will be assigned tasks to their strengths as ensure efficient use of skills and collaboration. Use project management tools such as Jira or Trello to communicate all task requirements, sprint details, and task completion in a collaborative manner to track contribution. Communicate and document using Microsoft Teams and regular sprint meetings and brief sprint reviews that will allow the team to talk about difficulties, developments, and possibilities for adjusting plans.

Gantt charts and task boards with task dependencies, deadlines, and milestones will be used to track progress. Incorporate risk management into the process, identifying potential technical or scheduling issues with a functionality and creating plans to mitigate the risk in advance.

Every step contains quality assurance: before connecting a module with another module or modules, unit level tests will be carried out on each module. Regular user tests will be conducted to ensure full functionality, usability, and accessibility. A final platform that satisfies student learning needs and exhibits educational outcomes will be created by incorporating the results of these user tests into the next development cycle.

This structured management approach anticipates the project being managed in a way that achieves timely completion and quality control, which enables a sustainable AI enabled platform for detecting learning weakness and adaptive guided learning in a real-time environment.

3.4 Feasibility Study

3.4.1 Technical Feasibility

The goal for this technical feasibility study regarding the AI-Powered Web Platform is to assess if the possible project can be developed using the current resources, tools and technologies.

The project will utilize modern web technologies, such as **MongoDB/MySQL** for secure and efficient data storage, Spring Boot or Node.js/Express for backend, and React.js for frontend. **Python** will be used to develop the AST-based Weakness Detection module using scikit-learn or Tensorflow, and ast to parse code. In the Code Concept Extraction module, NLTK or spaCy will be used for the natural language processing in the Code Concept Extraction module.

Real-time monitoring of student code, adaptive feedback, and gamified reinforcement are just a few of the possibilities offered by the technologies chosen. The **Monaco Editor** will allow the user to input code in the browser, compile it, and execute it in real-time with feedback being sent to both the AST Analyzer and the Weakness Detection modules simultaneously.

Overall, the project is technically feasible because of the access to all necessary tools, technologies and personnel available. The modular architecture promotes the development of an AI-powered learning platform that is scalable, maintainable and efficient.

3.4.2 Schedule Feasibility

The schedule feasibility assesses whether the AI-Powered Web Platform can be built within the anticipated timeframe, compromising function scope and complexity of the modules, and availability of the resources.

This project is planned over a **6–8 month**. Using Agile methodologies and tools for task and due date management such as Trello/Jira, we will track overall project progress. Project divided into clearly defined phases:

1. **Requirement Gathering and Analysis**– Identify the functional needs and integration points of my component with the main system. Gather requirements for the platform and specific component.
2. **System Design and Architecture**– Designing the modular architecture, database schemas, data flow diagrams, and component interactions.
3. **Module Development**– Implementation of the core modules. **Initially ,Code Input Environment** and the **AST Analyzer**, which together form the foundation for real-time weakness detection. Subsequent sprints will focus on the **Weakness Detection Engine**, ensuring accurate identification of coding mistakes, inefficiencies, and conceptual gaps.
4. **Testing and Integration**–integrating modules and carrying out performance, usability, integration, and unit tests.
5. **Deployment & Maintenance Plan** – Deploy the platform and establish a maintenance schedule. Performance optimization, bug fixes, and final adjustments.

3.4.3 Economic Feasibility

The economic viability of component relies principally on the cost/benefit analysis of the compiler component and development IDE (the Monaco Editor) as well as other tools and resources.

Since the component will initially integrate the Monaco Editor (as open-source) as the development IDE in place of a commercial IDE, it will not incur licensing costs, but it will still have a professional, rich feature coding environment. For compilation and execution of code, the system will incorporate multiple open-source compilers and/or interpreters (e.g., GCC/Clang, Node.js, or the native active runtimes for each programming language) that can also be available free of charge for integration and count as little to no real expense to the company for the base code writing/editing/execution.

The financial detail of the project centers on the cost minimization aspect of open source technologies coupled with what will effectively be open-source project management tools (i.e. Trello or Jira) owner-free tier plan or no cost. Furthermore, the development hardware could be established or vaguely manipulated from existing instruments, so as a project we avoided significant hardware purchases.

3.5 System Analysis

3.5.1 Software solution approach

The solution in the software to be suggested is an interactive, intelligent coding environment wherein students will write code and execute code but will also be coached along the way in iterative ways of learning and improving. The approach pulls several software modules into one, flexible system that is focused on adaptive learning and constant feedback.

1. **Code Input Environment**– Development in the integrated development environment powered by the Monaco Editor begins with students writing their code. The integrated development environment supports usability by providing syntax highlighting, autocomplete, and error checking.
2. **Abstract Syntax Tree (AST) Analyzer**-The submitted code is parsed into an Abstract Syntax Tree (AST) to allow the system to analyze the ability to have structural and conceptual programming. This is a necessary step for identifying logical flows, syntax structures, and the potential error sources.

3. **Weakness Detection Engine**– In order to find common mistakes, ineffective coding structures, or conceptual misunderstandings, the engine analyzes the AST using the relevant rules and/or machine learning models.
4. **Adaptive Feedback and Guided Learning**-The student receives contextual feedback in real time. While the Guided Learning Interface offers structured, step-by-step solutions when necessary or suggestions when appropriate, the Adaptive Feedback Module offers hints or corrections.
5. **Progress Profiling and Tracking**-Every student's interactions and coding weaknesses are tracked and recorded over time to create a Weakness Profile. Long-term data will assist in offering tailored adaptive learning pathways and suggestions for enhancement.
6. **Gamification Layer**-By adding points, badges, and levels, the system adds another level of gamification to increase motivation and engagement.

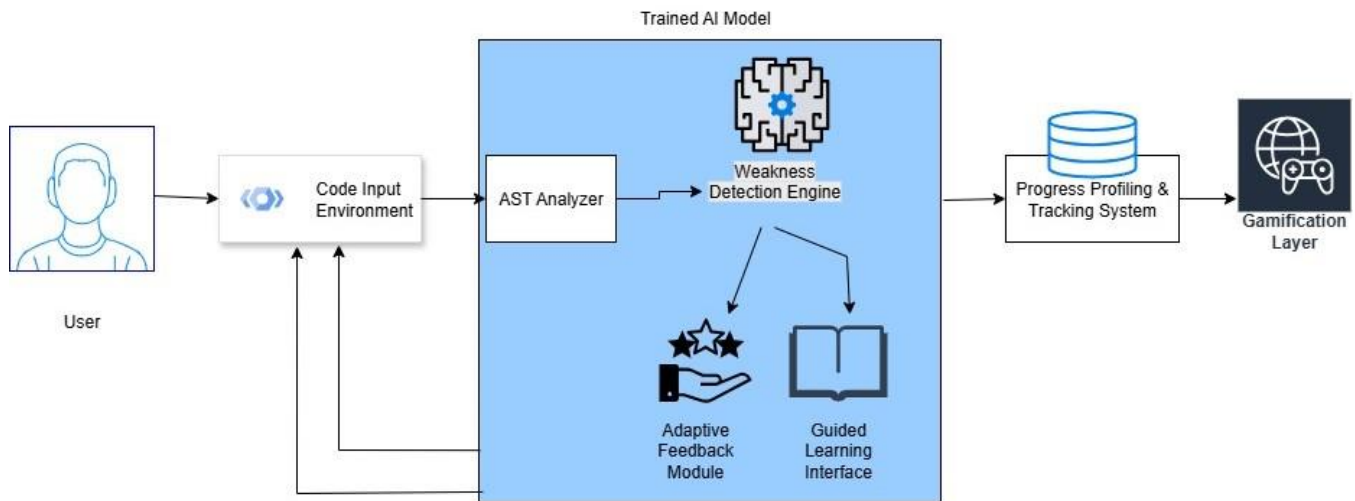


Figure 2: System Diagram

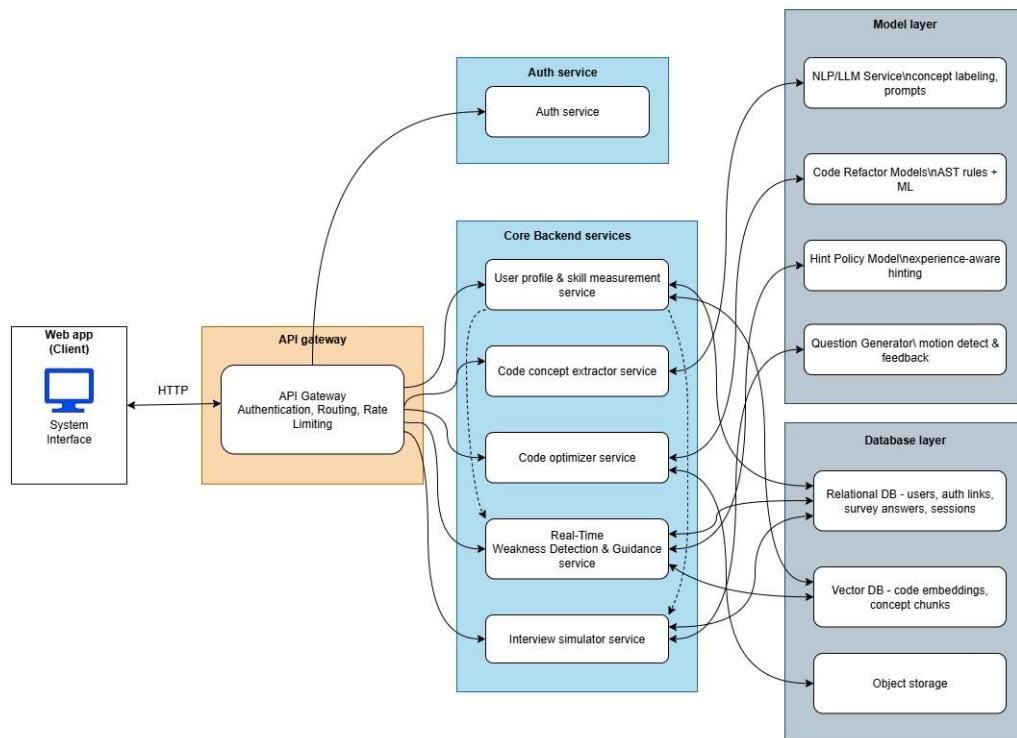


Figure 3: Overall System Diagram

3.5.2 Tools and technologies

The solution that is suggested combines the use of various tools and technologies to make sure that it has seamless functionality, scalability, and adaptability. These technologies are chosen well to balance out performance, usability and cost-effectiveness.

Monaco Editor-Provides the web-based Integrated Development Environment (IDE). Offers features such as syntax highlighting, IntelliSense, autocompletion, and error detection. Lightweight, customizable and the same editor as Visual Studio Code, familiar to students.

Compiler/ Interpreter Programming Language- Student-submitted code will be run in GCC/Clang (enforcement of C/C++ languages) and Node.js (execution of JavaScript languages). They are open-source, reliable and used in many academic and industrial environments.

Abstract Syntax Tree (AST) Parser- The AST representation is produced by a parsing library (e.g. Esprima to JavaScript, ANTLR to many languages).

Backend Framework- The code execution requests, AST analysis and communication between the frontend and back-end modules are handled by Node.js with Express.js. It is scalable and supports real-time feedback.

Database- Student data, weaknesses profiles, history of feedbacks and gamification development is stored in MongoDB. It is flexible with dynamic data and can support large numbers of learners.

Gamification and Feedback Module- Embedded with bespoke algorithms and embedded with frontend UI systems. Monitors progress, rewards points and shows progress badges in real time.

Frontend Technologies- React.js to create an interactive, responsive and modular interface. Monaco Editor integration to experience smooth coding.

Version Control and Collaboration Project version control-Team collaboration and codebase management is done through GIT and GitHub.

These tools and technologies collectively represent a powerful, low-priced and scalable platform to deliver interactive, smart programming education.

Project Management – Jira (sprint planning, task management)

3.6 Project requirements

3.6.1 Functional requirements

The expected behavior and operation of the system is described in the following functional requirements:

Code Editing and Input– The system should enable users (students) to write, edit and modify code through the built-in Monaco Editor. The editor must assist syntax coloring, auto-complete and error marks.

Code Compilation and Execution– The system should integrate and run code with its respective open-source compilers and runtimes (e.g. GCC/Clang, Node.js). It must show compilation errors and run time results in real time.

AST (Abstract Syntax Tree) Analysis– The system has to decode user code to an AST to be further analyzed. It has to identify code constructions, logical flow and programming patterns.

Weakness Detection–The system should point to typical errors in coding (e.g. logical errors, unproductive loops, recursion errors). It must give elaborate information on problem areas.

Adaptive Feedback Generation–The system should be able to give real time feedback depending on the weaknesses identified. It must have hints, corrective advice or best practice recommendations.

Guided Learning Support–The system should provide step by step instructions on how to solve complicated problems. It must respond to feedback depending on the student learning pace.

Progress Tracking and Profiling– The system should keep user progress and a performance profile. It must monitor the repeat errors and general progress overtime.

Gamification Features– Students have to be rewarded with points, badges, or levels by the system when they complete coding tasks. It must promote participation by rewarding it.

User Management– Students should be able to log in safely in the system. When registering to the system student should give their educational level(beginner, intermediate, advanced). Allow creation of individual student profiles for tracking progress and personalizing learning.

3.6.2 Non- Functional requirements

Performance – The system should be able to give real time code analysis and feedback with a low latency (preferably less than 2 seconds when submitting standard code). The system must be able to handle multiple users at the same time without markedly reducing the performance.

Scalability – The architecture needs to be scalable to support more and more students and submissions of codes. Extra modules or features need not be designed significantly without redesigning.

Usability – The site should have an easy to use interface and easy navigation. The functionality such as Monaco Editor must be conveniently located and user-friendly both to beginners and higher-level students.

Reliability and Availability – The system must not take much of the down time and must be in service at least 99 percent of the planned time. Student progress and performance data should be backed up on a regular basis.

Security – The system should safeguard the information of students, such as logins and the development history, through secure authentication and encryption. It must avoid unauthorized users of user accounts and code submissions.

Maintainability –The system must also be well documented and modular to be updated or debugged in future. Code ought to be best by best practices in both readability and maintainability.

Compatibility – It should be compatible with the most common web browsers (Chrome, Firefox, Edge) and gadgets (desktop, laptop, tablet).

Extensibility– Elements of future AI or programming languages, or gamification features should be able to be easily integrated with the platform without significant platform re-architecture.

Data Integrity– The system should be able to properly track and store the performance of students and feedback. There should be no inconsistency of data between modules due to data synchronization.

3.6.3 User requirements

Target Users– University students who are learning to program (beginner, intermediate, advanced).

User Goals– Learn to code better, learn programming, and get ready to show up at technical interviews.

User Interaction– toocode write, real-time feedback, guided exercises, progress tracking and gamification challenges.

User Constraints – Browser-based, limited to small-to-medium code snippets, no support for large-scale projects.

3.6.4 System requirements

Hardware requirements-

Standard desktop or laptop with internet access.

Software requirements-

Frontend: React.js with Monaco Editor

Backend: Node.js/Express.js (or Spring Boot)

Database: MongoDB or MySQL

AST Analysis: Python (TensorFlow / scikit-learn, ast library)

NLP: NLTK / spaCy

Browsers: Chrome, Firefox, Edge

3.6.5 Use case scenarios

Use case ID	UC001
Use case name	User Registration
Primary Actor	Student
Pre Conditions	User has stable internet connection

Main Flow	1. User opens the Web Platform that is AI-Powered. 2. Enter personal details (name, email) 3. Select a coding language given in the system. 4. Answer to questions given from the system.(system assign a skill level based on the answered questions.) 5. Clicks “Register”. 6. System develops a student profile and logs the user in.
Extensions	3a.Inaccurate information or invalid data to the system results in error messages.

Table 3:Use case scenario1

Use case ID	UC002
Use case name	Answer a question
Primary Actor	Student
Pre Conditions	Student is registered and logged in
Main Flow	1. System give task based on user’s skill level. 2. Student click start button. 3. System prompts a coding space. 4. Student start coding in the given code space.

Table 4:Use case scenario2

Use case ID	UC003
Use case name	Code Solution & Receive Adaptive Feedback
Primary Actor	Student

Pre Conditions	Student has submitted a coding question
Main Flow	1. Student types in Monaco Editor. 2. AST analysis is used by system monitors to code in real-time. 3. Feedback will be delivered on the level of the skills of the student: • Beginner → Comprehensive instructions and step-by-step guidance. • Intermediate → Partial hints and moderate guidance. • Advanced → Minimal hints, mostly suggestions. 4. Student updates code based on feedback. 5. System tracks mistakes and conceptual weaknesses for profiling. Extensions.
Extensions	2a. Compilation/runtime error → System points out errors and gives corrective clues. 3a. Network problem → The system caches code on its disk until the connection is fixed. 5a. Often used errors will cause customized micro-challenges or additional instructions.

Table 5: Use case scenario3

3.7 Project scope

The project aims at developing an AI-based web application where students of a university can train their coding skills, get adaptive feedback and identify their progress. The platform combines weakness detection modules, guided learning modules, progress profiling modules, and gamified engagement modules.

3.7.1 In-scope

The Monaco Editor allows students to write, compile and execute small to medium sized snippets of code in the browser.

Real-time AST-based weakness detection and adaptive feedback for code errors and conceptual mistakes.

Step wise problem-solving **guided learning** with hints based on skill level. Beginners will receive comprehensive instructions and step-by-step tips, Intermediates will receive partial hints and moderate guidance, Advanced will receive little direction.

Progress profiling and long-term tracking of weaknesses.

Badges, points and progress levels as **gamification practices** to boost motivation.

3.7.2 Out-scope

Support for **large-scale or multi-file projects**, due to browser-based performance and compilation limitations.

Additional IDE capabilities such as large application debugging, a comprehensive project versioning system or sharing editing.

Execution of code with **high memory or runtime environment** needs that is not supported by standard interpreter or compiler.

Integration with external enterprise systems or proprietary development environments.

3.8 Testing

The stage of testing is one of the essential steps in the process of making sure that the AI-Powered Web Platform works as desired. The system has the following testing strategy; unit testing, integration testing, usability testing, performance testing, gamification testing, and security testing. Its aim is to confirm the correct operation of each module, the seamless operation of the modules, and the provision of a smooth, secure and engaging learning experience to the students by the platform.

3.8.1 Unit Testing

Unit testing is used to confirm the functionality of the individual system components. Key unit tests for the platform include:

- **AST Analyzer:** Check that recursion errors, inefficient loops and base case errors are identified right in student code.
- **Adaptive Feedback Engine:** Hints and corrective suggestions are to be created based on the skill level (Beginner, Intermediate, Advanced).
- **Gamification Module:** Make sure that when tasks are done, the points, badges and levels are calculated correctly.
- **Code Compiler/Executor:** Test that submitted code is executed with the correct result and errors are returned with the correct result. Where feasible unit tests will be automated and run after every module is developed.

3.8.2 Integration Testing

- Test that Monaco Editor submissions that activate AST processing and produce adaptive feedback.
- Ensure that progress tracking points, badges and levels are updated.
- Savings and retrieving of user data and activity logs in the database are done correctly.
- Test smooth real-time communication between frontend and backend (<2 seconds).

3.8.3 Usability Testing

Make sure that the students can navigate Monaco Editor easily. Recognize clues and feedback manifest on-site and on-target with appropriate skill level. Check accessibility features like readable fonts, color contrast, and mobile responsiveness. Close the gap by collecting user feedback.

3.8.4 Performance Testing

Ensure code execution and feedback generation respond in under 2 seconds. Performance under multiple users (10 -50) at the same time submitting code. Evaluate system behavior under peak loads.

3.8.5 Gamification Testing

Check points, badges, and levels are updated instantly. Check that leaderboard and progress tracking are working. Make sure that gamification can be adjusted to the level of skills (Beginner, Intermediate, Advanced). Practice and improvement test rewards.

3.8.6 Security Testing

Verify secure login and registration with encrypted passwords. Store and transfer sensitive data encrypted. Test that only authorized users can access profiles and code. Check for vulnerabilities like SQL injection, XSS, and CSRF.

4. WORK BREAKDOWN

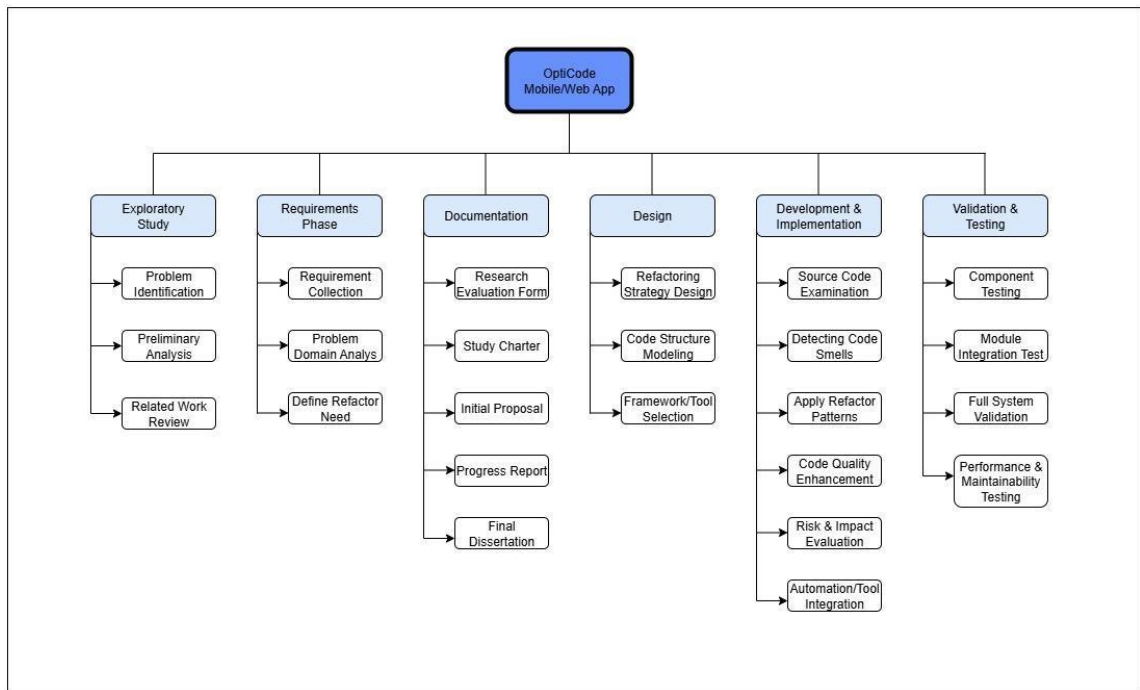


Figure 4: Work breakdown structure

4.1 TimeLine

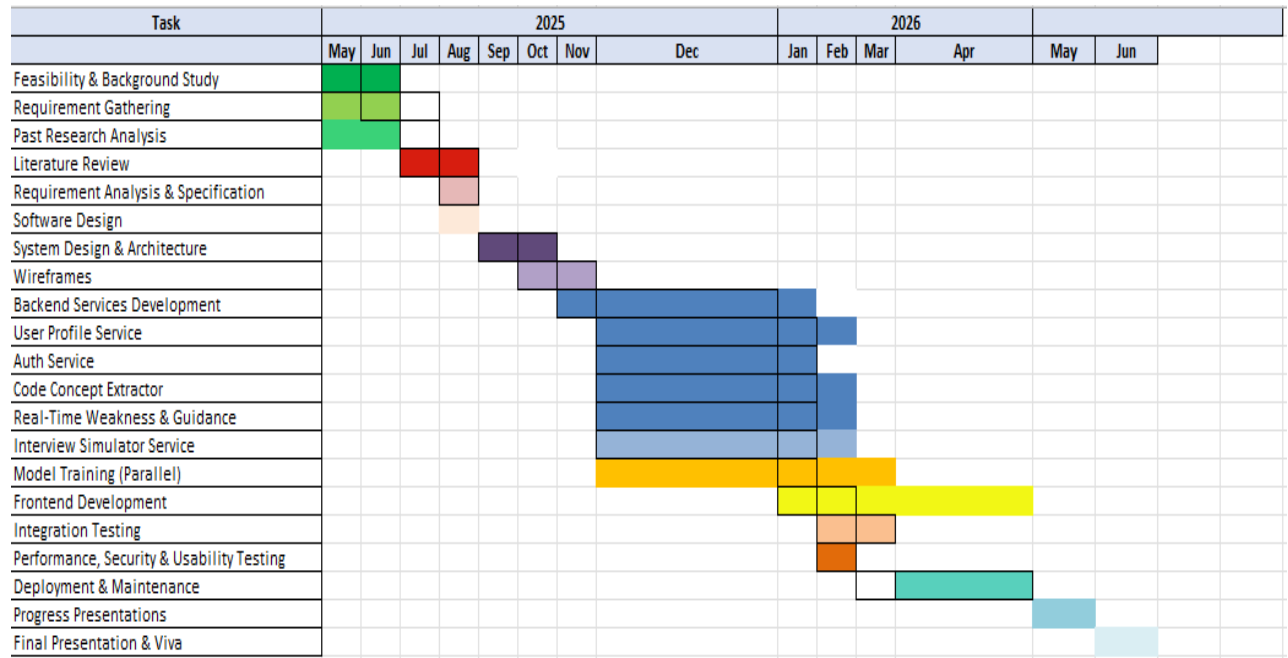


Figure 5: Gantt Chart

5. COMMERCIALIZATION

Target Market: University students and coding boot camps as well as e-learning systems in Sri Lanka and the world.

Revenue Model: Freemium Supplementary usage, the premium subscription (advanced analytics and interview preparation) LKR 500-1000/month. University students buy institutional licensing in bulk.

Unique Selling Points (USP): Dynamic weaknesses (not only syntax errors), detection. Gamified adaptive learning (11 levels, badges, points). Long-term developments monitoring + interview preparation. Scalability: Has the capacity to scale to other programming languages and be used with existing LMS.

Business Potential: There are over 1M+ students in South Asia who study programming; the company can collaborate with universities, coding bootcamps, and EdTech businesses.

6. BUDGET

1. Software & Tools (LKR 0 – 30,000):

Monaco Editor (open source, free of charge).

Open-source compilers (GCC, Node.js, Python runtimes – free of charge).

AI/ML libraries (TensorFlow, scikit-learn, spaCy, NLTK – free of charge).

Project Management tools (Jira free-tier, GitHub student pack – free/cheap).

2. Cloud & Hosting (LKR 30,000 – 50,000):

Cloud hosting (AWS/GCP/Azure free tier initially, ~LKR 10,000–15,000/month for scaling up).

Database hosting (MongoDB Atlas/MySQL cloud).

3. Development & Testing Resources (LKR 20,000 – 40,000):

Student laptops/PCs (assumed there is no extra cost as they are available, etc).

Internet & testing environment.

4. Miscellaneous (LKR 10,000 – 20,000):

Domain name, SSL certificate.

Type	Cost
Software & Tools	10,000
Cloud & Hosting	20,000
Development & Testing	15,000
Other	15,000
Total	60,000

Table 6: Cost management plan

7.REFERENCES

- [1] A. Robins, J. Rountree, and N. Rountree, "Learning and teaching programming: A review and discussion," *Computer Science Education*, vol. 13, no. 2, pp. 137– 172, 2003.
- [2] D. A. McCall and M. Kölling, "A new look at novice programmer errors," *ACM Transactions on Computing Education*, vol. 19, no. 4, pp. 1–35, 2019.
- [3] B. A. Becker, K. Goslin, and C. Denny, "Effective compiler error message enhancement for novice programmers," in *Proc. 16th Koli Calling Int. Conf. Comput. Educ. Res.*, 2016, pp. 3–11.
- [4] GitHub, "Introducing GitHub Copilot: Your AI pair programmer," *GitHub Blog*, Jun. 2021. [Online]. Available: <https://github.blog/2021-06-29-introducing-github-copilot/> [Accessed: Aug. 30, 2025].
- [5] OpenAI, "ChatGPT: Optimizing language models for dialogue," OpenAI, Nov. 2022. [Online]. Available: <https://openai.com/blog/chatgpt> [Accessed: Aug. 30, 2025].
- [6] N. Kiesler, "A secondary analysis of novice learners' steps and errors when writing recursive functions in Java," in *Proc. Psychology of Programming Interest Group (PPIG)*, 2022, pp. 45–56.

8.APPENDICE

8.1Plagerism Report

project proposal draft-Ashani Hettiarachchi-IT22604194.docx			
ORIGINALITY REPORT			
6%	5%	0%	4%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS
PRIMARY SOURCES			
1	Submitted to Sri Lanka Institute of Information Technology Student Paper	3%	
2	repository.riarauniversity.ac.ke:8080 Internet Source	1%	
3	www.coursehero.com Internet Source	<1%	
4	repository.nwu.ac.za Internet Source	<1%	
5	Submitted to University of Westminster Student Paper	<1%	
6	Submitted to University of Bolton Student Paper	<1%	
7	ijisrt.com Internet Source	<1%	
8	Submitted to Harrisburg University of Science and Technology Student Paper	<1%	
	fastercapital.com		
9	Internet Source	<1%	
10	refubium.fu-berlin.de Internet Source	<1%	
11	www.preprints.org Internet Source	<1%	

